SPLASH Team Progress Report 01

COMP 8117 Expert Review 01

Team members Abhinav Pandey (lead), Shrijana Karki, Aakriti Mainali and Khemraj Adhikari

Date [date]

Executive summary

Our project started in a different place than where it is now. Our first plan was about helping a startup scale its own AI model on rented GPUs during a traffic spike without paying for idle machines. We wrote a full plan for it with a budget and a load testing setup. When we discussed it with the professor and the GA on October 7 they raised three concerns that we now agree with. We would not be able to show real evidence because the startup was fictional and the traffic was simulated. The scope was not clear because the plan mixed a business case with a DevOps setup and a model optimization problem. And a real optimization result was not realistic in the time we have left.

Reading week ran from October 10 to 18 and most of it went to exams. During that week we still met to discuss the feedback. We talked about whether the original idea could work with a much narrower scope and a real stakeholder, and we looked at a number of other ideas. We used that feedback to step back and look for a problem where real people are affected today and where existing tools clearly fall short. After comparing several options we chose to work on lecture material that blind and low vision students cannot use properly.

In one word our problem is meaning. Lecture slides store pictures and text boxes but not what they mean. A screen reader or braille display can read text, but it cannot tell a blind student what a formula, a table or a diagram says when that content is saved as an unlabelled picture. Today a person fixes these files by hand, which we have read can take days.

Over the weekend of October 24 and 25 we researched existing work and ran a first feasibility check on a real lecture. The check confirmed the problem on real material and showed that adding the missing structure automatically is technically possible. It also showed us where the hard parts are. The biggest open questions are how accurate an automatic description step can be, how long conversions take at our own university, and how many real screen reader users we can test with.

Problem investigation

The professor asked us to find a practical problem with real impact where we could collect evidence. We looked at several ideas including border wait times in Windsor, bus arrival predictions and flaky tests in open source projects. Each time we found that a popular tool already solved most of it. Google Maps covers driving times and the Transit app and Umo already show live bus arrivals. Accessible course material was different because the existing tools each solve only one piece and the full path from an instructor file to a student who can understand it is still broken.

The people who feel the problem are blind and low vision students in regular university courses. The people who own the problem are the accessibility office and the university. The people who could change it are instructors who create the files and staff who convert them. The direct user of our software will most likely be accessibility staff or instructors, not the students. We learned from the SPLASH guide that these roles can be different people and in our case they clearly are.

We also had to learn how a file actually reaches a blind student, because at first we had it wrong. We assumed there was a special NVDA format that a PDF needs to be converted into. There is not. NVDA is a screen reader and it reads whatever the file contains. A PDF holds drawing instructions that say where text and pictures go on a page. It can also hold hidden tags that say what each part is and in which order to read it. The PDF viewer builds a list of items from those tags and the screen reader reads that list as speech. The same text goes to a braille display through Liblouis, and math is read properly through MathCAT if it is stored as MathML. When there are no tags the viewer guesses, and pictures with no description are announced only as a graphic or skipped. So the part that breaks is the file itself. Everything after it already works well.

We also looked at braille devices because they seemed at first like they might already solve the problem. The APH and HumanWare Monarch is a braille tablet with about 3,840 pins. It shows braille and simple tactile pictures and runs its own apps. According to the APH Monarch FAQ it opens braille files, EPUB and DAISY books, PDF and Word documents, and images, and supports MathML. But it does not turn a normal document into proper braille by itself, and an image is raised in pins as it is. APH says simple images work best. A detailed lecture slide squeezed into roughly 96 by 40 dots cannot be read by touch. Someone still has to simplify diagrams and describe formulas first, so the device depends on the same missing meaning.

Some things are still assumptions. We assume conversion at our university takes days. We assume diagrams and math are the worst problem areas. We assume staff would use a tool that does most of the work and flags what it is unsure about. We plan to test these through interviews with Student Accessibility Services and instructors.

Information and evidence

Before deciding anything we needed to know how blind students read, what already exists, and whether a team like ours could add something real.

For how screen readers and braille work we used documentation from NV Access who make NVDA, the WCAG guidelines from W3C, material on MathML and MathCAT, and the APH and HumanWare pages and FAQ for the Monarch. These are the sources the field itself relies on so we trust them for how things should work.

For existing tools we found that Adobe Acrobat auto tagging, Equidox, CommonLook and DocAble already add tags or convert math automatically. Their claims come from the vendors themselves. Equidox for example claims to remove most of the manual tagging work. We found no independent

test of how well they do on lecture slides, so we treat these claims with caution and plan to test them ourselves.

For research we looked at several papers. Work on AI remediation of STEM PDFs presented at ACM ASSETS 2022 adds structure and formulas automatically but still makes tagging mistakes on complex layouts. A 2025 paper on step by step remediation of scientific PDFs still has a person set the reading order by hand. A study comparing human and AI written descriptions of computer science figures found that automatic quality scores miss the errors that matter most to screen reader users, which directly shaped how we plan to evaluate. FigurA11y, AlternAtIve and AltGosling use AI to describe scientific figures but their tests with blind users are small, in one case a single user. Olli and Chart Reader from MIT give layered descriptions that start with an overview and then go into detail, but only for charts. VizAbility lets blind users ask questions about a chart. Recent work on automatic tactile graphics is still early and lists refreshable pin displays as future work.

The sources agree on what makes a file accessible. They disagree on how well automatic tools work, with vendors sounding confident and researchers finding errors that matter. Taken together the pieces exist but separately and mostly for papers or charts. We found nobody who combines them for lecture slides, checks the accuracy of the descriptions, and tests the result through what a screen reader actually receives. We have not read every paper in full yet, so we treat this gap as our current understanding and not a final claim.

Feasibility check

We wanted to know whether the problem is real on actual lecture material and whether fixing it is technically possible. We ran everything locally on our own laptop.

We took a public lecture, Stanford CS231n Lecture 2, which has 98 slides. Checking the file showed it has no tags and no structure tree at all. Slide 23 explains a distance formula with a worked example of three 4 by 4 grids of numbers adding up to 456. The formula and the grids are stored as two pictures with no description. The only text a screen reader can get from that slide is the title, the words L1 distance and add, and the footer. The formula, all 48 numbers and the answer are missing. On the slide about the linear classifier the screen reader gets scraps of the equation and matrix sizes in the wrong order with nothing linking them. On the nearest neighbour slide it gets the words deer bird plane cat car as a plain list while the six photos and the main idea are silent. We put these three slides side by side with what a screen reader gets in a comparison page.

We then built a small prototype to see if the missing meaning can be added back. It reads the slide title from the PDF, takes a layered description of each slide with an overview, its parts and how they relate, turns formulas written in LaTeX into MathML, and builds a page with real headings, lists, tables and math. It can also export that page as a tagged PDF. In this first run the descriptions, formulas and table values were written by Claude looking at the slide images, standing in for the automatic description step we still need to build.

We checked the result using the accessibility tree that Chrome builds, which is what a screen reader receives. The original slides gave no headings, no readable formulas, no tables and no descriptions. Our version gave 17 headings to move between, 4 formulas a screen reader can speak, 3 real tables with all 48 numbers readable cell by cell, and 3 layered descriptions. One result surprised us. When we exported the same page as a tagged PDF the headings, lists and tables kept their tags but the formulas did not get a formula tag, so the math meaning was lost. This supports our decision to make HTML the main output.

So the honest answer to whether this is possible is partly yes. Adding structure, tables and speakable math works. The hard part that is still unproven is producing correct descriptions automatically, and that is where our engineering work will focus.

The check also has limits. We read Chrome's accessibility tree, not real NVDA speech, because NVDA only runs on Windows. We have not yet run the slides through Adobe or Equidox for a fair comparison. And no blind user has tried the result yet.

How we will measure evidence

Recruiting many blind users in a few weeks will be hard, so we plan to measure in layers.

The first layer is automated checks of the original and converted files.

The second layer is what a screen reader actually says. NVDA on Windows and VoiceOver on Mac can both be controlled from code with a testing library called Guidepup, which records everything the screen reader speaks. This lets us show the real spoken output of the original file next to ours.

The third layer is an AI agent that only receives what the screen reader speaks and never sees the slide. We give it realistic tasks such as finding a formula, explaining a diagram or reading a value from a table, and record whether it succeeds.

The fourth layer is accuracy. People check a sample of the descriptions and we report how often they are wrong.

The fifth layer is real people. We will run a smaller number of sessions with screen reader users or accessibility staff using the same tasks and compare their results with the agent. We only trust the agent as far as it agrees with real users.

Engineering strategy

We chose to start with a short investigation phase and then move into one week iterations. Our biggest risk is misunderstanding the problem, not the technology. The weekend research already changed our plans twice. We dropped the idea of building braille output once we learned existing tools handle it, and we chose HTML over tagged PDF after seeing the math lost in our own test. Short iterations let us keep making changes like these based on what we find.

We considered a plan driven waterfall approach and rejected it because our requirements are still uncertain. We also considered full Scrum but with four part time students and about five weeks of build time we felt the ceremonies would cost more than they save. We keep a weekly planning meeting, a short weekly review and a visible task board.

Each week we pick the assumption or feature that matters most, build or test it, measure the result and decide what to do next. Important decisions go into a decisions log with what we decided, the options, the evidence and what would make us change it. The pivot from the GPU project, the decision not to build braille output and the choice of HTML are the first entries.

Team and development environment

Our team has four members. Abhinav Pandey is the team lead and the other members are Shrijana Karki, Aakriti Mainali and Khemraj Adhikari. We have not assigned specific roles yet. We plan to discuss them as a team once the investigation shows which parts of the work need the most effort, and we will record the decision in the decisions log.

Everyone writes code, reviews someone elses pull requests and joins interviews when possible. Deployment will be scripted to a single command with a short guide so members without DevOps experience can do it.

We use GitHub for code and the decisions log, GitHub Projects as our task board and pull requests with at least one reviewer. Interview notes and test files go in a shared drive. NVDA needs Windows, so one member will keep a Windows machine or virtual machine ready.

The prototype code from the feasibility check exists and will go into the repository. The repository and task board are [set up or not yet set up]. The Guidepup screen reader setup is not done yet.

Planning and resources

Our goal is to show that the time and effort needed to give a blind student a usable version of a lecture can be reduced in a measurable way. Our scope is lecture slides with math and diagrams turned into accessible HTML. We are not building braille output, not handling every document type, not building a full product for students and not making tactile pictures for the Monarch unless we can borrow a device. If our investigation shows a different content type matters more we will change scope and record why.

In week 6 we contact the accessibility office, collect real lecture files and run them through Adobe auto tagging for a fair baseline. In week 7 we finish interviews and automate the description step. In week 8 we get Guidepup and the agent evaluation working. In weeks 9 and 10 we convert real files, measure accuracy, run user sessions and compare before and after. Week 11 is for analysis and reports and week 12 is the final defense.

The main resource is our time. We estimate each member can give about 8 to 10 hours a week, roughly 200 to 250 hours in total, and we will log hours weekly. Money costs are small, about 50 to

100 dollars for AI model use and hosting. NVDA and the tools we used for the feasibility check are free. We also depend on time from accessibility staff and test participants which we do not control.

Quality risks and controls

The qualities that matter most are whether a screen reader user can understand the converted file, whether the descriptions are correct and whether the pipeline gives consistent results. We check these through the layers of evidence above, tests on every pull request and a weekly manual review of a sample of descriptions.

Our success criteria are a large increase in what a screen reader can access, a clear increase in tasks completed, conversion on the same day instead of days, and a description error rate we will set after measuring the baseline. We set exact targets before building so the results can still prove us wrong.

The main risks are these. The accessibility office may not reply soon, so we are emailing this week and asking instructors directly too. Automatic descriptions may be wrong more often than we hope, so we measure accuracy and flag uncertain output for a person. We may find very few real users, so we use the screen reader and agent method for scale and confirm it with even three to five people. Commercial tools may turn out better than we expect on slides, in which case we report that and narrow our focus to what they miss. If the scope turns out too big we drop tables first and keep math and diagrams.

Current state and next steps

So far we finished our first plan and received feedback from the professor and the GA on October 7. During reading week from October 10 to 18 we discussed a narrower version of the original idea and other alternatives, and then chose a new direction. Over the weekend of October 24 and 25 we researched existing tools, devices and papers, ran a feasibility check on a real lecture, built a first prototype and measured its output. We also wrote a proposal and problem definition.

We have not yet confirmed the problem at our own university, interviewed anyone, automated the description step, compared against Adobe or Equidox, or recorded real NVDA speech.

Our next steps are to email Student Accessibility Services, collect real lecture files from two or three instructors, run them through Adobe auto tagging, and set up Guidepup.

We would like feedback on a few questions. Is converting one material type into HTML a reasonable scope for the time we have left? Is screen reader output with an AI agent an acceptable form of evidence if we also validate it with a small number of real users? How many real user sessions would make our results convincing? Should we treat the accessibility office as a stakeholder, as the main user of our tool, or both?

Supporting artifacts

Original project plan for GPU scaling.

Proposal and problem definition for accessible lecture slides, including the feasibility check and prototype results.

Comparison page of three real slides showing what a screen reader gets.

Prototype code and the accessibility tree output in the prototype folder of our repository.

Decisions log in docs/decisions-log.md in our repository.

Repository https://github.com/Abhinavxox/Splash-Project

Use of AI tools

We used Claude to help research existing tools and papers, to build the first prototype and to help draft this report. In the prototype Claude also wrote the slide descriptions by hand as a stand in for the automatic step. The team reviewed all claims and decisions and remains responsible for them. We still need to read the main papers in full before citing them.